

Table of Contents

Introduction	1
1. <code>docker</code> CLI	2
1.1 Container Related Commands	2
1.2 Image Related Commands	4
1.3 Network Related Commands	5
1.4 Registry Related Commands	6
1.5 Volume Related Commands	6
1.6 All Related Commands	6
2. Dockerfile	6
About the Authors	8

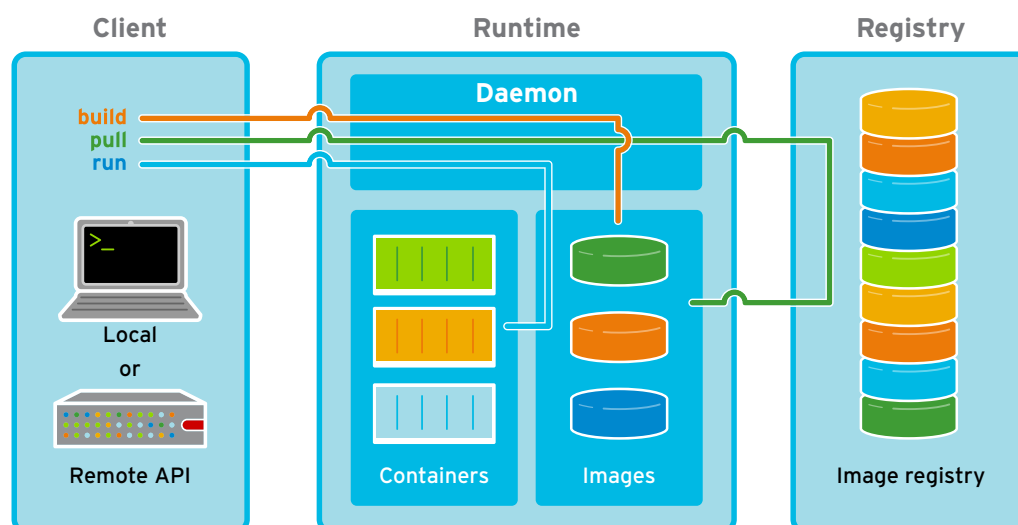
Introduction

Containers allow the packaging of your application (and everything that you need to run it) in a “container image”. Inside a container you can include a base operating system, libraries, files and folders, environment variables, volume mount-points, and your application binaries.

A “container image” is a template for the execution of a container – It means that you can have multiple containers running from the same image, all sharing the same behavior, which promotes the scaling and distribution of the application. These images can be stored in a remote registry to ease the distribution.

Once a container is created, the execution is managed by the container runtime. You can interact with the container runtime through the “`docker`” command. The three primary components of a container architecture (client, runtime, & registry) are diagrammed below:

Container Architecture



1. docker CLI

1.1 Container Related Commands

`docker [CMD] [OPTS] [CONTAINER]`

Examples

All examples shown work in Red Hat Enterprise Linux

1. Run a container in interactive mode:

#Run a bash shell inside an image

```
$ docker run -it rhel7/rhel bash
```

#Check the release inside a container

```
[root@.../]# cat /etc/redhat-release
```

2. Run a container in detached mode:

```
$ docker run --name mywildfly -d -p 8080:8080 jboss/wildfly
```

3. Run a detached container in a previously created container network:

```
$ docker network create mynetwork
```

```
$ docker run --name mywildfly-net -d --net mynetwork \
    -p 8080:8080 jboss/wildfly
```

4. Run a detached container mounting a local folder inside the container:

```
$ docker run --name mywildfly-volume -d \
    -v myfolder:/opt/jboss/wildfly/standalone/deployments/ \
    -p 8080:8080 jboss/wildfly
```

5. Follow the logs of a specific container:

```
$ docker logs -f mywildfly
```

```
$ docker logs -f [container-name|container-id]
```

6. List containers:

List only active containers

```
$ docker ps
```

List all containers

```
$ docker ps -a
```

7. Stop a container:

Stop a container

```
$ docker stop [container-name|container-id]
```

Stop a container (timeout = 1 second)

```
$ docker stop -t1
```

8. Remove a container:

Remove a stopped container

```
$ docker rm [container-name|container-id]
```

Force stop and remove a container

```
$ docker rm -f [container-name|container-id]
```

Remove all containers

```
$ docker rm -f $(docker ps -aq)
```

Remove all stopped containers

```
$ docker rm $(docker ps -q -f "status=exited")
```

9. Execute a new process in an existing container:

Execute and access bash inside a WildFly container

```
$ docker exec -it mywildfly bash
```

Command	Description
<code>daemon</code>	Run the persistent process that manages containers
<code>attach</code>	Attach to a running container to view its ongoing output or to control it interactively
<code>commit</code>	Create a new image from a container's changes
<code>cp</code>	Copy files/folders between a container and the local filesystem
<code>create</code>	Create a new container
<code>diff</code>	Inspect changes on a container's filesystem
<code>exec</code>	Run a command in a running container
<code>export</code>	Export the contents of a container's filesystem as a tar archive
<code>kill</code>	Kill a running container using SIGKILL or a specified signal
<code>logs</code>	Fetch the logs of a container
<code>pause</code>	Pause all processes within a container
<code>port</code>	List port mappings, or look up the public-facing port that is NAT-ed to the PRIVATE_PORT
<code>ps</code>	List containers
<code>rename</code>	Rename a container
<code>restart</code>	Restart a container
<code>rm</code>	Remove one or more containers
<code>run</code>	Run a command in a new container
<code>start</code>	Start one or more containers
<code>stats</code>	Display one or more containers' resource usage statistics
<code>stop</code>	Stop a container by sending SIGTERM then SIGKILL after a grace period
<code>top</code>	Display the running processes of a container
<code>unpause</code>	Unpause all processes within a container
<code>update</code>	Update configuration of one or more containers
<code>wait</code>	Block until a container stops, then print its exit code

1.2 Image Related Commands

`docker [CMD] [OPTS] [IMAGE]`

Examples

All examples shown work in Red Hat Enterprise Linux

1. Build an image using a Dockerfile:

#Build an image

```
$ docker build -t [username/]<image-name>[:tag] <dockerfile-path>
```

#Build an image called myimage using the Dockerfile in the same folder where the command was executed

```
$ docker build -t myimage:latest .
```

2. Check the history of an image:

Check the history of the jboss/wildfly image

```
$ docker history jboss/wildfly
```

Check the history of an image

```
$ docker history [username/]<image-name>[:tag]
```

3: List the images:

```
$ docker images
```

4: Remove an image from the local registry:

```
$ docker rmi [username/]<image-name>[:tag]
```

5. Tag an image:

Creates an image called "myimage" with the tag "v1" for the image jboss/wildfly:latest

```
$ docker tag jboss/wildfly myimage:v1
```

Creates a new image with the latest tag

```
$ docker tag <image-name> <new-image-name>
```

Creates a new image specifying the "new tag" from an existing image and tag

```
$ docker tag <image-name>[:tag][username/] <new-image-name>[:new-tag]
```

6. Exporting and importing an image to an external file:

Export the image to an external file

```
$ docker save -o <filename>.tar
```

Import an image from an external file

```
$ docker load -i <filename>.tar
```

7 Push an image to a registry:

```
$ docker push [registry/][username/]<image-name>[:tag]
```

Command	Description
<code>build</code>	Build images from a Dockerfile
<code>history</code>	Show the history of an image
<code>images</code>	List images
<code>import</code>	Create an empty filesystem image and import the contents of the tarball into it
<code>info</code>	Display system-wide information
<code>inspect</code>	Return low-level information on a container or image
<code>load</code>	Load an image from a tar archive or STDIN
<code>pull</code>	Pull an image or a repository from the registry
<code>push</code>	Push an image or a repository to the registry
<code>rmi</code>	Remove one or more images
<code>save</code>	Save one or more images to a tar archive (streamed to STDOUT by default)
<code>search</code>	Search one or more configured container registries for images
<code>tag</code>	Tag an image into a repository

1.3 Network related commands

`docker network [CMD] [OPTS]`

Command	Description
<code>connect</code>	Connects a container to a network
<code>create</code>	Creates a new network with the specified name
<code>disconnect</code>	Disconnects a container from a network
<code>inspect</code>	Displays detailed information on a network
<code>ls</code>	Lists all the networks created by the user
<code>rm</code>	Deletes one or more networks

1.4 Registry related commands

Default is `https://index.docker.io/v1/`

Command	Description
<code>login</code>	Log in to a container registry server. If no server is specified then default is used
<code>logout</code>	Log out from a container registry server. If no server is specified then default is used

1.5 Volume related commands

```
docker volume [CMD] [OPTS]
```

Command	Description
create	Create a volume
inspect	Return low-level information on a volume
ls	Lists volumes
rm	Remove a volume

1.6 Other commands

Command	Description
events	Get real time events from the server
inspect	Show version information
docker version	Show the docker CLI version

2. Dockerfile

The Dockerfile provides the instructions to build a container image through the `docker build -t [username/]<image-name>[:tag] <dockerfile-path>` command. It starts from a previously existing Base image (through the FROM clause) followed by any other needed Dockerfile instructions.

This process is very similar to a compilation of a source code into a binary output, but in this case the output of the Dockerfile will be a container image.

Example Dockerfile

This example creates a custom WildFly container with a custom administrative user. It also exposes the administrative port 9990 and binds the administrative interface publicly through the parameter 'bmanagement'.

```
# Use the existing WildFly image
FROM jboss/wildfly

# Add an administrative user
RUN /opt/jboss/wildfly/bin/add-user.sh admin Admin#70365 --silent

#Expose the administrative port
EXPOSE 8080 9990

#Bind the WildFly management to all IP addresses
CMD ["/opt/jboss/wildfly/bin/standalone.sh", "-b", "0.0.0.0",
"-bmanagement", "0.0.0.0"]
```

Using the example Dockerfile

```
# Build the WildFly image
$ docker build -t mywildfly .

#Run a WildFly server
$ docker run -it -p 8080:8080 -p 9990:9990 mywildfly

#Access the WildFly administrative console and log in with the credentials admin/Admin#70635
open http://<docker-daemon-ip>:9990 in a browser
```

Dockerfile instruction arguments

Command	Description
FROM	Sets the base image for subsequent
MAINTAINER	Sets the author field of the generated images
RUN	Execute commands in a new layer on top of the current image and commit the results
CMD	Allowed only once (if many then last one takes effect)
LABEL	Adds metadata to an image
EXPOSE	Informs container runtime that the container listens on the specified network ports at runtime
ENV	Sets an environment variable
ADD	Copy new files, directories, or remote file URLs from into the filesystem of the container
COPY	Copy new files or directories into the filesystem of the container
ENTRYPOINT	Allows you to configure a container that will run as an executable
VOLUME	Creates a mount point and marks it as holding externally mounted volumes from native host or other containers
USER	Sets the username or UID to use when running the image
WORKDIR	Sets the working directory for any RUN, CMD, ENTRYPOINT, COPY, and ADD commands
ARG	Defines a variable that users can pass at build-time to the builder using --build-arg
ONBUILD	Adds an instruction to be executed later, when the image is used as the base for another build
STOPSIGNAL	Sets the system call signal that will be sent to the container to exit

Basic example

```
# docker-compose.yml
version: '2'

services:
  web:
    build:
      # build from Dockerfile
      context: ./Path
      dockerfile: Dockerfile
    ports:
      - "5000:5000"
    volumes:
      - ./code
  redis:
    image: redis
```

Version 1 to 2

Docker compose is now integrated into the official Docker installation. The functionality only improved over that change, and the simple syntax change is : V1 : docker-compose ARG to V2 docker compose ARG More on that here : [Docker Compose Migrate to V2](#)

Commands

<code>docker compose version</code> <code>docker compose config</code>
<code>docker compose start</code> <code>docker compose stop</code> <code>docker compose restart</code> <code>docker compose run</code>
<code>docker compose create</code> <code>docker compose attach</code> <code>docker compose pause</code> <code>docker compose unpause</code>
<code>docker compose wait</code> <code>docker compose up</code> <code>docker compose down</code>
<code>docker compose ps</code> <code>docker compose top</code> <code>docker compose events</code> <code>docker compose logs</code>
<code>docker compose images</code> <code>docker compose build</code> <code>docker compose push</code> <code>docker compose cp</code> <code>docker compose exec</code>

Reference

Building

```
web:
  # build from Dockerfile
  build: .
  args:      # Add build arguments
    APP_HOME: app

  # build from custom Dockerfile
  build:
    context: ./dir
    dockerfile: Dockerfile.dev

  # build from image
  image: ubuntu
  image: ubuntu:14.04
  image: tutum/influxdb
  image: example-registry:4000/postgresql
  image: a4bc65fd
```

Ports

```
ports:
  - "3000"
  - "8000:80" # host:container

# expose ports to linked services (not to host)
expose: ["3000"]
```

Commands

```
# command to execute
command: bundle exec thin -p 3000
command: [bundle, exec, thin, -p, 3000]

# override the entrypoint
entrypoint: /app/start.sh
entrypoint: [php, -d, vendor/bin/phpunit]
```

Environment variables

```
# environment vars
environment:
  RACK_ENV: development
environment:
  - RACK_ENV=development

# environment vars from file
env_file: .env
env_file: [.env, .development.env]
```

Dependencies

```
# makes the `db` service available as the host
# (implies depends_on)
links:
  - db:database
  - redis

# make sure `db` is alive before starting
depends_on:
  - db

# make sure `db` is healthy before starting
# and db-init completed without failure
depends_on:
  db:
    condition: service_healthy
  db-init:
    condition: service_completed_successfully
```

Other options

```
# make this service extend another
extends:
  file: common.yml # optional
  service: webapp

volumes:
  - /var/lib/mysql
  - ./_data:/var/lib/mysql

# automatically restart container
restart: unless-stopped
# always, on-failure, no (default)
```

Advanced features

Labels

```
services:
  web:
    labels:
      com.example.description: "Accounting web"
```

DNS servers

```
services:
  web:
    dns: 8.8.8.8
  dns:
    - 8.8.8.8
    - 8.8.4.4
```

Devices

```
services:
  web:
    devices:
      - "/dev/ttyUSB0:/dev/ttyUSB0"
```

External links

```
services:
  web:
    external_links:
      - redis_1
      - project_db_1:mysql
```

Healthcheck

```
# declare service healthy when `test` comma
healthcheck:
  test: ["CMD", "curl", "-f", "http://local
  interval: 1m30s
  timeout: 10s
  retries: 3
  start_period: 40s
```

Hosts

```
services:
  web:
    extra_hosts:
      - "somehost:192.168.1.100"
```

Network

```
# creates a custom network called `frontend`
networks:
  frontend:
```

External network

```
# join a pre-existing network
networks:
  default:
    external:
      name: frontend
```

Volume

```
# mount host paths or named volumes, specified
db:
  image: postgres:latest
  volumes:
    - "/var/run/postgres/postgres.sock:/var/r
    - "dbdata:/var/lib/postgresql/data"

volumes:
  dbdata:
```

User

```
# specifying user
user: root
```

```
# specifying both user and group with ids
user: 0:0
```